HOSPITAL EMERGENCY RESPONSE CO-ORDINATOR

SOURCE CODE

Resource Process

```
#include "hospital_messages.h"


int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    name_attach_t *attach = name_attach(NULL, ATTACH_POINT_RESOURCE, 0);
    if (attach == NULL) {
        perror("[ResourceProcess] name_attach failed");
        return EXIT_FAILURE;
    }

    printf("[ResourceProcess] Online and listening on '%s' (chid=%d)\n",
           ATTACH_POINT_RESOURCE, attach->chid);
    fflush(stdout);

    int dashboard_coid = -1;
    hospital_msg_t msg;
    hospital_reply_t rep;
    int rcvid;

    int icu_beds = 5;
    int general_beds = 20;
    int outpatient_slots = 50;
```

```
  while (1) {
    rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);
    if (rcvid == -1) {
      perror("[ResourceProcess] MsgReceive error");
      continue;
    }

    /* Pulse or disconnect notification */
    if (rcvid == 0) {
      continue;
    }

    if (msg.msg_type == MSG_TYPE_BATCH_DONE) {
      printf("[ResourceProcess] Batch complete signal received. Forwarding to
Dashboard...\n");
      fflush(stdout);

      if (dashboard_coid == -1) {
        dashboard_coid = name_open(ATTACH_POINT_DASHBOARD, 0);
      }
      if (dashboard_coid != -1) {
        hospital_reply_t fwd_rep;
        MsgSend(dashboard_coid, &msg, sizeof(msg), &fwd_rep,
sizeof(fwd_rep));
      }
```

```
            memset(&rep, 0, sizeof(rep));
            rep.msg_type = MSG_TYPE_REPLY_OK;
            rep.status = 0;
            strncpy(rep.message, "Resource batch forwarded", sizeof(rep.message) -
1);
            MsgReply(rcvid, EOK, &rep, sizeof(rep));
            continue;
        }

        if (msg.record.priority.level == PRIORITY_P1_CRITICAL) {
            if (icu_beds > 0) icu_beds--;
            strncpy(msg.record.resource.allocated_dept, "ICU",
sizeof(msg.record.resource.allocated_dept) - 1);
            strncpy(msg.record.resource.status_str, "ICU treatment started",
sizeof(msg.record.resource.status_str) - 1);
            msg.record.resource.allocated = 1;
        } else if (msg.record.priority.level == PRIORITY_P2_HIGH ||
msg.record.priority.level == PRIORITY_P3_MEDIUM) {
            if (general_beds > 0) general_beds--;
            strncpy(msg.record.resource.allocated_dept, "GENERAL BED",
sizeof(msg.record.resource.allocated_dept) - 1);
            strncpy(msg.record.resource.status_str, "General bed assigned",
sizeof(msg.record.resource.status_str) - 1);
            msg.record.resource.allocated = 1;
        } else {
            if (outpatient_slots > 0) outpatient_slots--;
```

```
        strncpy(msg.record.resource.allocated_dept, "OUTPATIENT",
sizeof(msg.record.resource.allocated_dept) - 1);
        strncpy(msg.record.resource.status_str, "Outpatient slot assigned",
sizeof(msg.record.resource.status_str) - 1);
        msg.record.resource.allocated = 1;
    }

    printf("[ResourceProcess] Patient %s assigned to %s (Remaining ICU:%d,
Gen:%d, Out:%d)\n",
           msg.record.patient.patient_id,
           msg.record.resource.allocated_dept,
           icu_beds, general_beds, outpatient_slots);
    fflush(stdout);
    if (dashboard_coid == -1) {
        dashboard_coid = name_open(ATTACH_POINT_DASHBOARD, 0);
        if (dashboard_coid == -1) {
            printf("[ResourceProcess] Warning: Could not connect to '%s' yet
(errno=%d). Retrying...\n",
                   ATTACH_POINT_DASHBOARD, errno);
            fflush(stdout);
        }
    }
    if (dashboard_coid != -1) {
        msg.msg_type = MSG_TYPE_DASHBOARD_POST;
        hospital_reply_t dash_rep;
```

```
        if (MsgSend(dashboard_coid, &msg, sizeof(msg), &dash_rep,
sizeof(dash_rep)) == -1) {
            perror("[ResourceProcess] MsgSend to DashboardProcess failed");
            name_close(dashboard_coid);
            dashboard_coid = -1;
        }
    }
    memset(&rep, 0, sizeof(rep));
    rep.msg_type = MSG_TYPE_REPLY_OK;
    rep.status = 0;
    strncpy(rep.message, "Resource Allocated & Forwarded",
sizeof(rep.message) - 1);
    MsgReply(rcvid, EOK, &rep, sizeof(rep));
  }

  name_detach(attach, 0);
  return EXIT_SUCCESS;
}
```